



RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DE CARTHAGE
Institut National des Sciences Appliquées et de Technologie

TRAITEMENT D'IMAGES – GL4

Reconnaissance Automatique des Émotions Faciales

Rapport Final

Réalisé par :

Leith ENGAZZOU
Takoua AYADI
Hiba CHABBOUH
Mohamed Sahbi BEN KALAIA
Yassine DAHMOUL

Année Universitaire : 2025 – 2026

Table des matières

I	Cadrage du Projet	4
1	Problématique	4
1.1	Contexte général	4
1.2	Problème à résoudre	4
1.3	Enjeux et applications	4
1.4	Défis scientifiques et techniques	5
2	Objectifs du Projet	6
2.1	Objectif principal	6
2.2	Objectifs spécifiques	6
2.3	Livrables attendus	6
3	Description des Données	7
3.1	Présentation du dataset FER2013	7
3.2	Structure et répartition des classes	7
3.3	Qualité et limites	7
3.4	Accès et formats	7
II	Prétraitement des Données	8
1	Introduction	8
2	Analyse du Dataset FER2013	8
2.1	Distribution des classes et déséquilibre	8
2.2	Exemples de données brutes	10
3	Méthodes de Prétraitement	10
3.1	Vue d'ensemble du pipeline	10
3.2	Conversion en niveaux de gris et redimensionnement	11
3.3	Estimation du niveau de bruit	11
3.4	Débruitage adaptatif	11
3.5	Égalisation d'histogramme adaptative — CLAHE	11
3.6	Normalisation Z-score	12
4	Comparaison des Pipelines	13
5	Augmentation des Données	15
5.1	Transformations géométriques	15
5.2	Transformations photométriques	15
5.3	Régularisation par masquage — Random Erasing	15
6	Bilan du Prétraitement	15
III	Architectures et Choix des Modèles	16
1	Introduction et Contexte	16
2	Vue d'Ensemble des Approches	16

2.1	Approche Machine Learning — Baselines	16
2.2	Approche Deep Learning — Modèles de production	16
3	Architecture du CustomCNN	17
3.1	Philosophie de conception	17
3.2	Description détaillée des blocs	17
3.3	Résumé de l’architecture	17
4	Transfer Learning — VGG16 et ResNet50	18
4.1	Justification du transfer learning	18
4.2	Adaptations nécessaires	18
4.3	Stratégie d’entraînement en deux phases	18
4.4	Récapitulatif des choix architecturaux	18
IV	Entraînement et Analyse des Résultats	19
1	Introduction	19
2	Entraînement des Modèles ML Classiques	19
2.1	Hyperparamètres des modèles	19
2.2	Résultats de performance	20
2.3	Courbes d’apprentissage et de perte	20
2.4	Matrices de confusion	21
3	Custom CNN	23
3.1	Protocole d’entraînement	23
3.2	Analyse des courbes d’apprentissage	24
3.3	Métriques finales	25
3.4	Analyse par classe	25
3.5	Matrice de confusion	26
4	Transfer Learning — ResNet50	27
4.1	Présentation et configuration	27
4.2	Stratégie d’entraînement par phases	27
4.3	Courbes d’apprentissage et résultats	28
4.4	Analyse par classe et matrice de confusion	29
5	Transfer Learning — VGG16	30
5.1	Présentation et configuration	30
5.2	Courbes d’apprentissage et analyse	31
5.3	Matrice de confusion	32
5.4	Limites et recommandations	32
V	Interface de Démonstration Interactive	33
1	Motivation et Objectifs	33
2	Architecture de l’Application	33
2.1	Panneau latéral — Sélection des modèles	34
2.2	Zone principale — Inférence et résultats	34
3	Pipeline d’Inférence	34
3.1	Chemin ML classique	34
3.2	Chemin Deep Learning	35
4	Chargement Robuste des Checkpoints	36
5	Exemples d’Inférence	36
5.1	Cas favorable — Émotion nettement exprimée	37
5.2	Cas ambigu — Peur	37
5.3	Comparaison ML vs Deep Learning	38
6	Bilan Technique	38

VI Bilan Comparatif et Conclusion	39
1 Synthèse comparative des performances	39
2 Analyse critique par famille d'approches	39
3 Enseignements transversaux	40
4 Perspectives	40

CHAPITRE I

Cadrage du Projet

1 Problématique

1.1 Contexte général

La capacité à reconnaître et à interpréter les émotions humaines est fondamentale dans les interactions sociales [3]. Les êtres humains transmettent une grande part de leurs états internes au travers des expressions faciales, et cette communication non verbale joue un rôle central dans la compréhension mutuelle [1].

Avec l'essor de l'intelligence artificielle et de la vision par ordinateur, il devient envisageable de doter les machines d'une capacité analogue [2]. La Reconnaissance Automatique des Émotions Faciales (RAEF) constitue aujourd'hui un domaine de recherche actif, porté en particulier par les progrès des réseaux de neurones convolutifs (CNN) et des architectures d'apprentissage profond.

1.2 Problème à résoudre

Comment concevoir un système automatique capable de reconnaître, à partir d'une image fixe ou d'un flux vidéo, les sept émotions faciales de base — colère, dégoût, peur, joie, tristesse, surprise et neutralité — avec une précision et une robustesse suffisantes pour envisager des déploiements dans des conditions réelles ?

1.3 Enjeux et applications

Les domaines susceptibles de bénéficier d'un tel système sont nombreux et variés :

- **Éducation** : analyse de l'engagement et de l'attention des apprenants dans des contextes EdTech.
- **Santé** : aide à la communication pour les personnes atteintes de troubles du spectre autistique (TSA).
- **Commerce et marketing** : amélioration de l'expérience utilisateur et détection de l'insatisfaction.
- **Sécurité routière** : surveillance de la fatigue et du niveau d'attention du conducteur.
- **Médecine** : détection de la douleur ou du stress en contexte clinique.
- **IHM** : développement d'interfaces plus intuitives et adaptatives.

1.4 Défis scientifiques et techniques

Malgré les progrès récents, plusieurs obstacles demeurent :

- **Variabilité inter-individuelle** : les expressions faciales varient sensiblement selon les cultures, l'âge et le contexte de capture.
- **Occultations** : lunettes, masques ou mains peuvent masquer des régions faciales clés et dégrader les performances.
- **Conditions d'acquisition** : variations d'éclairage, de pose ou de résolution.
- **Ambiguïté émotionnelle** : la frontière entre certaines émotions — peur et surprise notamment — est intrinsèquement floue.
- **Déséquilibre des données** : la sous-représentation de certaines classes impose des stratégies d'entraînement adaptées.

2 Objectifs du Projet

2.1 Objectif principal

L’ambition centrale de ce projet est de concevoir, implémenter et évaluer un pipeline complet de RAEF, couvrant l’ensemble de la chaîne depuis le prétraitement des images jusqu’à la classification finale par apprentissage profond.

2.2 Objectifs spécifiques

Objectifs essentiels (MVP). — Mise en place du pipeline de prétraitement, implémentation d’un CNN de base sur FER2013, atteinte d’une accuracy $> 60\%$ et analyse des erreurs via matrice de confusion.

Objectifs intermédiaires. — Exploration du *Transfer Learning* avec VGG16, ResNet50 et MobileNetV2, détection faciale automatique avec OpenCV ou MTCNN, et application de techniques de régularisation.

Objectifs avancés. — Développement d’une interface interactive (Streamlit), comparaison systématique des architectures retenues et interprétabilité des décisions via Grad-CAM.

2.3 Livrables attendus

Thème	Contenu principal
Cadrage (CR1)	Problématique, objectifs, description du dataset
Prétraitement (CR2)	Méthodes de traitement, illustrations avant/après
Modèle (CR3)	Architecture CNN, pipeline global
Entraînement (CR4)	Courbes loss/accuracy, analyse des hyperparamètres
Évaluation (CR5)	Métriques de performance, analyse critique
Finalisation (CR6)	Rapport final, démonstration, soutenance

TABLE I.1 – Planification des livrables du projet

3 Description des Données

3.1 Présentation du dataset FER2013

Le dataset FER2013, introduit lors de la compétition ICML 2013 [1], constitue le benchmark de référence pour ce projet. Il rassemble environ 35 887 images de visages en niveaux de gris, toutes à une résolution de 48×48 pixels, réparties en sept catégories émotionnelles.

3.2 Structure et répartition des classes

Le dataset est organisé en trois ensembles distincts : *train*, *publictest* (validation) et *privatetest* (test). La répartition adoptée après un split reproductible (graine 42, proportion 80/20 sur les données d'entraînement) est la suivante :

Ensemble	Nombre d'images	Proportion
Train	22 968	64,1 %
Validation	5 741	16,0 %
Test	7 178	20,0 %
Total	35 887	100 %

La distribution par classe est la suivante :

Code	Émotion	Nb. Images (approx.)	Observations
0	Colère	3 995	Bien représentée
1	Dégoût	436	Classe très minoritaire — rapport 1 :16,5 avec Joie
2	Peur	4 097	Souvent confondue avec la surprise
3	Joie	7 215	Classe majoritaire
4	Tristesse	4 830	Expression subtile
5	Surprise	3 171	Souvent confondue avec la peur
6	Neutre	4 965	État de référence

TABLE I.2 – Distribution des classes dans FER2013

Le déséquilibre marqué entre les classes — en particulier la classe « Dégoût » — impose le recours à des techniques compensatoires telles que la *data augmentation* ciblée ou le *class weighting*.

3.3 Qualité et limites

FER2013 présente plusieurs limites inhérentes : faible résolution (48×48 px), diversité ethnique limitée, bruit de labellisation estimé à environ 15 % selon la littérature, et un *domain gap* notable entre les photos collectées automatiquement et les conditions du monde réel.

3.4 Accès et formats

Le dataset est disponible via Kaggle en format CSV ou sous forme de dossiers d'images organisés par classe [7].

CHAPITRE II

Prétraitement des Données

1 Introduction

Un prétraitement rigoureux des données constitue une condition nécessaire à la convergence des modèles et à leur robustesse face à la variabilité naturelle des images. Cette phase couvre la normalisation, le débruitage et l’augmentation des données brutes de FER2013 avant leur injection dans les réseaux de neurones.

Le pipeline mis en place enchaîne les étapes suivantes :

- Conversion en niveaux de gris et redimensionnement à 48×48 pixels
- Débruitage adaptatif (filtre bilatéral ou Non-Local Means selon le niveau de bruit estimé)
- Égalisation d’histogramme adaptative **CLAHE**
- Normalisation Z-score ($\mu = 0,5071$, $\sigma = 0,2520$)
- Augmentations de données appliquées à l’entraînement uniquement
- Gestion du déséquilibre de classes par pondération

2 Analyse du Dataset FER2013

2.1 Distribution des classes et déséquilibre

La figure II.1 illustre le déséquilibre marqué entre les classes : la classe *Joie* représente plus de 25 % du dataset (7 215 images), tandis que *Dégoût* n’en contient que 436, soit un rapport de 1 à 16,5.

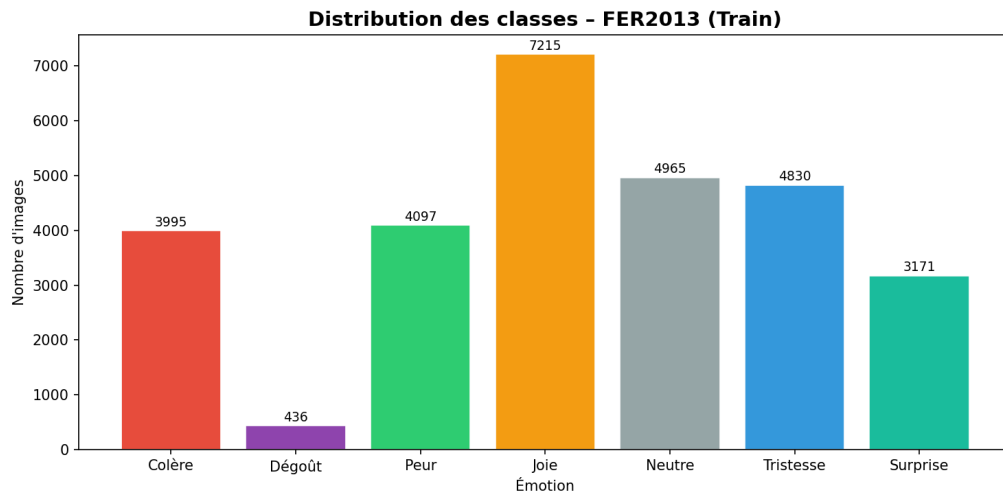


FIGURE II.1 – Distribution des classes dans le jeu d'entraînement FER2013.

Ce déséquilibre est traité par le calcul de **poids de classes inversement proportionnels** :

$$w_c = \frac{N}{C \times n_c}$$

où N est le nombre total d'images, C le nombre de classes, et n_c l'effectif de la classe c .

Émotion	Effectif	Poids
Colère	3 995	0,4800
Dégoût	436	4,3982
Peur	4 097	0,4681
Joie	7 215	0,2658
Neutre	4 965	0,3862
Tristesse	4 830	0,3970
Surprise	3 171	0,6047

Ces poids, maintenus constants pendant tout l'entraînement, sont transmis au paramètre `weight` de `nn.CrossEntropyLoss` : toute erreur sur une classe rare est ainsi proportionnellement plus pénalisée qu'une erreur sur une classe majoritaire.

2.2 Exemples de données brutes



FIGURE II.2 – Grille d'exemples bruts (avant prétraitement) pour chaque émotion.

La figure II.2 illustre la variabilité importante des images brutes : différences d'illumination, de cadrage, présence de filigranes et, dans quelques cas, des images corrompues (cadre entièrement noir).

3 Méthodes de Prétraitement

3.1 Vue d'ensemble du pipeline

Le pipeline est modulaire et propose trois niveaux d'intensité :

Pipeline	Étapes	Usage recommandé
minimal	CLAHE uniquement	Images peu bruitées
light	Débruitage adaptatif + CLAHE	Recommandé (défaut)
aggressive	NLM forcé + CLAHE	Images fortement dégradées

3.2 Conversion en niveaux de gris et redimensionnement

Bien que la majorité des images FER2013 soient déjà en niveaux de gris, certaines sont encodées en RGB avec trois canaux identiques. La transformation `Grayscale (num_output_channels=1)` harmonise l’ensemble en forçant un canal unique, réduisant la consommation mémoire d’un facteur 3 et simplifiant l’architecture du réseau.

Le redimensionnement à 48×48 pixels via `Resize` joue un rôle de garde-fou : les images FER2013 étant nativement à cette résolution, aucune interpolation n’est effectuée en pratique, mais cette étape garantit que toute image de taille inattendue soit automatiquement ramenée au format attendu.

3.3 Estimation du niveau de bruit

Avant d’appliquer un filtre, il est nécessaire d’estimer l’intensité du bruit afin d’éviter de dégrader des images propres. L’approche retenue consiste à soustraire une version légèrement lissée de l’image à l’original ; ce qui reste après soustraction est principalement du bruit :

$$\hat{\sigma}_{\text{bruit}} = \text{std}(I - G_{\sigma=1}(I))$$

Deux seuils empiriques calibrés sur FER2013 orientent le choix du filtre : en dessous de 50 (image propre), entre 50 et 150 (bruit modéré, filtre bilatéral), au-delà de 150 (bruit élevé, Non-Local Means).

3.4 Débruitage adaptatif

Filtre bilatéral. — Le filtre bilatéral améliore le filtre gaussien classique en ajoutant une condition de similarité en intensité : deux pixels ne sont moyennés que s’ils sont à la fois proches spatialement *et* similaires en intensité. Les contours nets (sourcils, lèvres) sont ainsi préservés. Paramètres : diamètre $d = 5$, $\sigma_{\text{color}} = 50$, $\sigma_{\text{space}} = 50$.

Non-Local Means (NLM). — Plus puissant que le filtre bilatéral, le NLM parcourt l’intégralité de l’image à la recherche de régions de texture similaire. Chaque pixel est remplacé par la moyenne pondérée de tous les pixels dont le voisinage ressemble au sien, une propriété particulièrement utile sur les surfaces uniformes des visages. Paramètres : force $h = 10$, fenêtre template 7×7 , fenêtre de recherche 21×21 .

3.5 Égalisation d’histogramme adaptative — CLAHE

La CLAHE (Contrast Limited Adaptive Histogram Equalization) découpe l’image en petites tuiles (4×4) et applique l’égalisation indépendamment dans chacune, améliorant le contraste localement sans surexposer les zones déjà claires. Le paramètre `clipLimit=1.5` écrête les pics de l’histogramme local pour éviter d’amplifier le bruit résiduel. La CLAHE est appliquée après le débruitage pour ne pas amplifier le bruit avant filtrage.

3.6 Normalisation Z-score

La normalisation Z-score ramène chaque pixel dans une plage centrée autour de zéro, ce qui stabilise la descente de gradient lors de l'entraînement :

$$\hat{x} = \frac{x - \mu}{\sigma} \quad \text{avec} \quad \mu = 0,5071, \sigma = 0,2520$$

Ces statistiques sont calculées sur l'ensemble de FER2013 après `ToTensor`. En pratique, la vérification sur un batch confirme une moyenne $\approx 0,07$ et un écart-type $\approx 1,16$.



FIGURE II.3 – Effet de la normalisation Z-score : image brute (gauche), après `ToTensor` (centre), après `Normalize` (droite).

4 Comparaison des Pipelines



FIGURE II.4 – Comparaison des trois pipelines pour une image représentative de chaque émotion.

L'observation des résultats conduit aux conclusions suivantes :

- Le pipeline *minimal* améliore sensiblement le contraste mais laisse le bruit intact.
- Le pipeline *light* offre le meilleur compromis : réduction du bruit préservant les contours faciaux importants pour la classification.
- Le pipeline *aggressive* produit des images trop lisses : certains détails discriminants (rides d'expression, dents) sont effacés.



FIGURE II.5 – Original vs. Prétraité (pipeline *light*, zoom $\times 6$). Ligne du haut : images brutes. Ligne du bas : après Grayscale $\rightarrow$ Resize $\rightarrow$ Débruitage adaptatif $\rightarrow$ CLAHE $\rightarrow$ Normalisation.

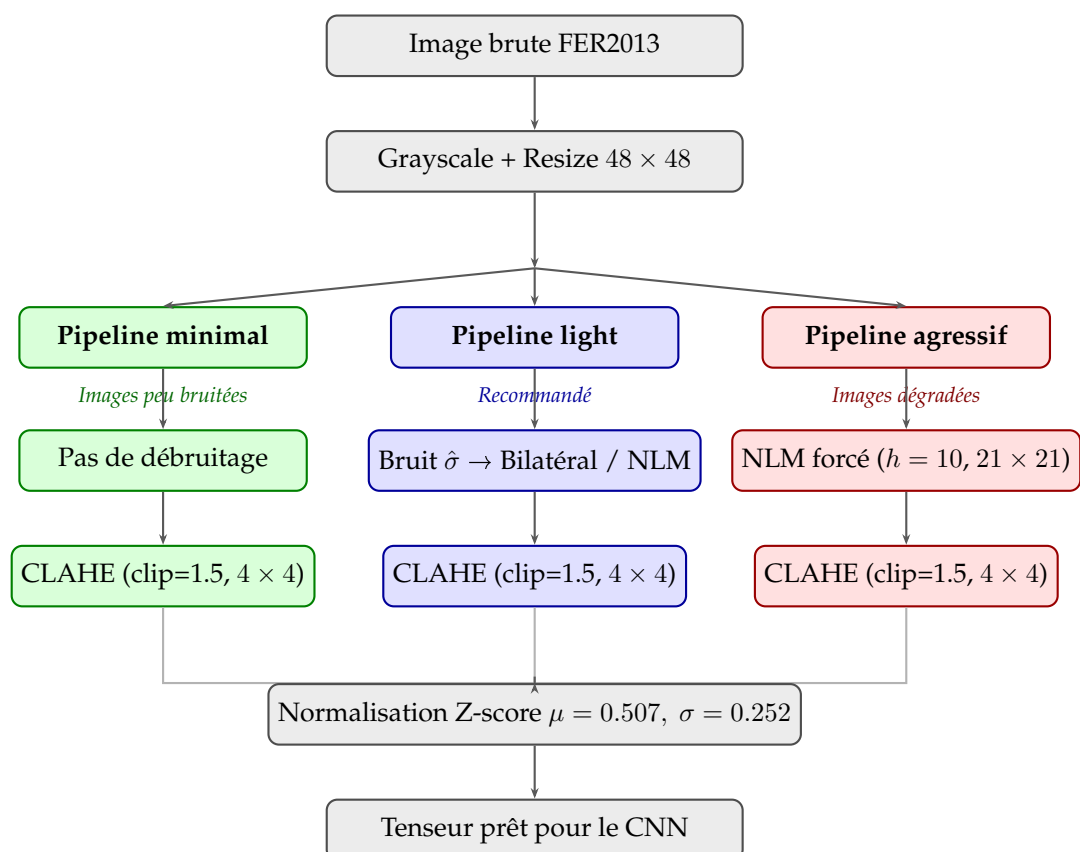


FIGURE II.6 – Vue schématique des trois pipelines de prétraitement.

5 Augmentation des Données

L’augmentation est appliquée **uniquement** au pipeline d’entraînement, à la volée, afin d’éviter toute fuite de données vers la validation.

5.1 Transformations géométriques

- **Flip horizontal** ($p = 0,5$) : les émotions étant symétriques, cette transformation double effectivement la taille exploitable du dataset.
- **Rotation aléatoire** (± 10) : compense les légères inclinaisons de tête présentes dans les photos naturelles.
- **Affine aléatoire** : translation de $\pm 5\%$ et mise à l’échelle dans $[0,95; 1,05]$, simulant des variations de cadrage caméra.

5.2 Transformations photométriques

- **ColorJitter** : variations de luminosité et de contraste dans $[0,7; 1,3]$, indispensable compte tenu des importantes variations d’illumination présentes dans FER2013.
- **RandomAdjustSharpness** ($p = 0,3$) : renforce la robustesse face aux images légèrement floues.

5.3 Régularisation par masquage — Random Erasing

`RandomErasing` ($p = 0,2$, surface $[2\%; 8\%]$) simule des occultations partielles du visage (lunettes, mains, masques), constituant une forme de *dropout* spatial qui atténue le surapprentissage.

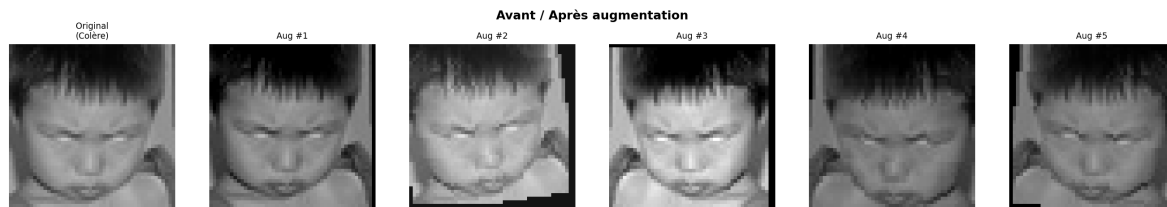


FIGURE II.7 – Effet de l’augmentation : image originale (Colère) et cinq versions augmentées produites à la volée lors de l’entraînement.

6 Bilan du Prétraitement

Étape	Justification	Paramètre clé
Grayscale	FER2013 est intrinsèquement en niveaux de gris	1 canal
Resize	Taille uniforme requise par les réseaux	48×48
Débruitage	Adaptatif : préserve les détails utiles	$\sigma_{\text{low}} = 50, \sigma_{\text{high}} = 150$
CLAHE	Contraste local sans amplification du bruit	$\text{clip}=1,5$
Normalisation	Stabilise l’entraînement	$\mu = 0,507, \sigma = 0,252$
Augmentation	Réduit le surapprentissage	Flip, Rotation, Jitter, Erasing
Poids classes	Compense le déséquilibre (Dégoût : 436 vs Joie : 7215)	$w_c \propto 1/n_c$

CHAPITRE III

Architectures et Choix des Modèles

1 Introduction et Contexte

Après avoir établi le pipeline de prétraitement, cette phase porte sur la sélection et la conception des modèles à entraîner sur FER2013.

Contraintes spécifiques du dataset

FER2013 impose des contraintes architecturales particulières : images de petite taille (48×48 pixels), canal unique (niveaux de gris), variations d'éclairage importantes, visages parfois mal centrés ou partiellement occultés, et fort déséquilibre entre classes (436 images pour Dégoût contre 7 215 pour Joie).

La stratégie adoptée compare deux familles d'approches : des modèles de **Machine Learning classique** servant de références (*baselines*), et des architectures de **Deep Learning** destinées à la production.

2 Vue d'Ensemble des Approches

2.1 Approche Machine Learning — Baselines

Trois modèles classiques sont entraînés sur les pixels aplatis (*flattened*) des images 48×48 :

- **Régression Logistique** : modèle linéaire multinomial, rapide à entraîner, servant de borne inférieure de performance.
- **Random Forest** : ensemble d'arbres de décision capturant certaines non-linéarités sans coût computationnel élevé.
- **SVC** (*Support Vector Classification*) : performant en grande dimension ($48 \times 48 = 2\,304$ features).

La limite fondamentale de ces approches réside dans le traitement de chaque pixel de façon *indépendante* : la relation spatiale entre pixels voisins — contour d'un sourcil, coin de la lèvre — est totalement ignorée.

2.2 Approche Deep Learning — Modèles de production

Trois architectures Deep Learning sont retenues :

- **CustomCNN** : réseau convolutif conçu spécifiquement pour les images 48×48 en niveaux de gris, entraîné depuis zéro.

- **VGG16** : architecture séquentielle profonde pré-entraînée sur ImageNet, adaptée par *fine-tuning*.
- **ResNet50** : architecture résiduelle pré-entraînée sur ImageNet, adaptée par *transfer learning* progressif.

3 Architecture du CustomCNN

3.1 Philosophie de conception

Le CustomCNN est conçu sur-mesure pour les contraintes de FER2013 : résolution 48×48 , canal unique, et risque élevé de surapprentissage sur un dataset de taille modérée. L'architecture suit une **progression pyramidale** : à mesure que la résolution spatiale diminue, le nombre de filtres augmente, permettant de construire des représentations de plus en plus abstraites.

3.2 Description détaillée des blocs

Chaque bloc convolutif suit la structure : deux convolutions 3×3 encadrées par BatchNorm et ReLU, suivies d'un MaxPooling 2×2 et d'un Dropout2d.

Bloc 1. — Entrée $48 \times 48 \times 1 \rightarrow$ sortie $24 \times 24 \times 64$. Détection des structures de bas niveau : bords, contrastes locaux, textures de peau.

Bloc 2. — Entrée $24 \times 24 \times 64 \rightarrow$ sortie $12 \times 12 \times 128$. Combinaison des détecteurs élémentaires en structures plus complexes : coins, formes partielles.

Bloc 3. — Entrée $12 \times 12 \times 128 \rightarrow$ sortie $6 \times 6 \times 256$. Encodage de patterns de haut niveau : configurations de sourcils, forme de la bouche, plissement des yeux.

La BatchNorm stabilise l'entraînement en normalisant les activations de chaque mini-batch, tandis que le Dropout2d(0.25) annule des *feature maps* entières pour forcer des représentations plus robustes.

Tête de classification

Après aplatissement ($6 \times 6 \times 256 = 9\,216$ valeurs), la tête fully connected comprend :

- Linear($9216 \rightarrow 512$) + BatchNorm + ReLU
- Dropout(0.5) — régularisation forte avant la décision finale
- Linear($512 \rightarrow 7$) — projection sur les 7 classes d'émotions

3.3 Résumé de l'architecture

Étape	Entrée	Sortie	Filtres	Params
Bloc 1 (Conv×2 + Pool)	$48 \times 48 \times 1$	$24 \times 24 \times 64$	64	~37K
Bloc 2 (Conv×2 + Pool)	$24 \times 24 \times 64$	$12 \times 12 \times 128$	128	~148K
Bloc 3 (Conv×2 + Pool)	$12 \times 12 \times 128$	$6 \times 6 \times 256$	256	~590K
Tête FC	9 216	7	—	~4,7M
Total	$\approx 5,87\text{M paramètres}$			

4 Transfer Learning — VGG16 et ResNet50

4.1 Justification du transfer learning

VGG16 [5] et ResNet50 [4] ont été pré-entraînés sur ImageNet (1,2 million d’images, 1 000 classes). Leurs premières couches ont appris des détecteurs génériques — bords, textures, formes — directement utiles pour la reconnaissance de visages. Réentraîner ces architectures depuis zéro serait coûteux et risqué au regard de la taille de FER2013.

4.2 Adaptations nécessaires

Les images FER2013 (48×48 , 1 canal) ne correspondent pas au format ImageNet (224×224 , RGB). Les adaptations suivantes sont appliquées :

- Redimensionnement à 224×224 par interpolation bilinéaire.
- Réplication du canal gris sur les 3 canaux RGB.
- Normalisation avec les statistiques ImageNet.
- Remplacement du classifieur final par une tête adaptée à 7 classes.

4.3 Stratégie d’entraînement en deux phases

L’entraînement suit une stratégie de *fine-tuning* progressif en deux phases pour éviter de détruire les features ImageNet dès le début.

Phase 1 — Warm-up (15 époques)

Le backbone est entièrement gelé ; seul le nouveau classifieur est entraîné avec Adam ($LR = 10^{-3}$). Cette phase initialise le classifieur avant de toucher au backbone.

Phase 2 — Fine-tuning (10 époques)

Les dernières couches du backbone sont dégelées (`layer4` pour ResNet50, blocs supérieurs pour VGG16) et entraînées avec SGD à faible taux ($LR = 10^{-4}$) pour affiner l’extraction de features sans effacer les connaissances bas-niveau.

4.4 Récapitulatif des choix architecturaux

Modèle	Avantages	Inconvénients
Logistic Reg.	Très rapide, interprétable	Linéaire, ignore les structures spatiales
Random Forest SVC	Robuste, peu de réglage Bon en grande dimension	Performances plafonnées sur images Lent, pas de fine-tuning
CustomCNN	Adapté à 48×48 , léger ($\sim 5,87$ M param.)	Apprend depuis zéro
VGG16 ResNet50	Features ImageNet robustes Meilleur compromis profondeur/efficacité	Très lourd (~ 138 M param.) Nécessite 224×224

CHAPITRE IV

Entraînement et Analyse des Résultats

1 Introduction

Ce chapitre présente les résultats expérimentaux obtenus en entraînant l'ensemble des modèles retenus sur FER2013. Pour chaque famille d'approche — Machine Learning classique, CustomCNN, ResNet50 et VGG16 — sont analysées les courbes d'apprentissage, les matrices de confusion et les métriques de performance finales.

- **Approches classiques (baseline)** : Régression Logistique, Random Forest et SVM à noyau RBF, entraînés sur les pixels bruts (48×48 aplatis), permettent d'évaluer les limites d'une représentation purement pixellique.
- **Approches profondes** : CNNs exploitant la structure spatiale des images, avec ou sans poids préentraînés sur ImageNet.

2 Entraînement des Modèles ML Classiques

2.1 Hyperparamètres des modèles

TABLE IV.1 – Hyperparamètres des modèles ML entraînés

Modèle	Paramètre	Valeur
Logistic Regression	solver	saga
	max_iter	1 000
	C	1,0 (régularisation L2)
Random Forest	n_estimators	200
	max_depth	30
	min_samples_split	5
SVM (RBF)	kernel	rbf
	C	10
	gamma	scale
	Entraînement	Sous-ensemble (contrainte computationnelle)

2.2 Résultats de performance

TABLE IV.2 – Résultats comparatifs des modèles ML sur FER2013

Modèle	Accuracy (%)	Precision macro (%)	Recall macro (%)	Temps (min)
Logistic Regression	≈ 35	≈ 34	≈ 32	4,99
Random Forest	≈ 46	≈ 57	≈ 40	0,56
SVM (RBF)	≈ 41	≈ 45	≈ 37	3,49

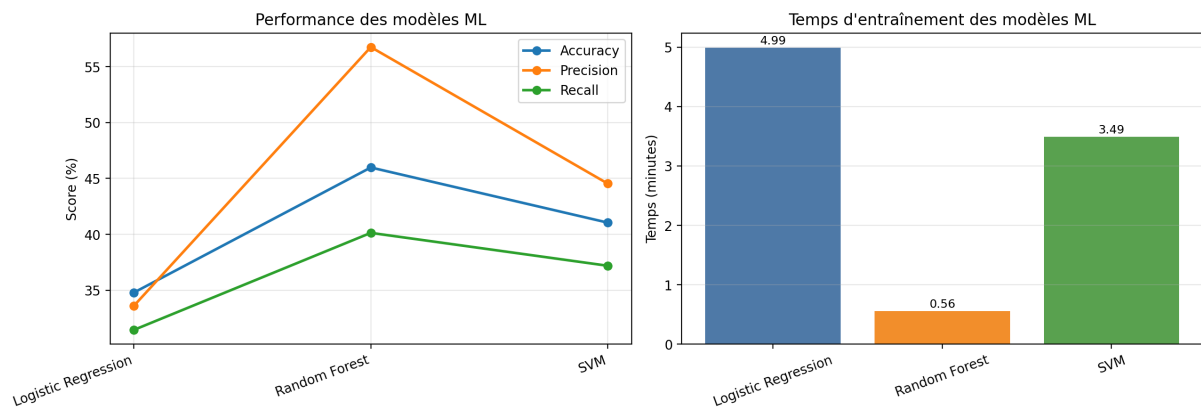


FIGURE IV.1 – Résumé des performances et des temps d'entraînement des trois modèles ML sur FER2013.

2.3 Courbes d'apprentissage et de perte

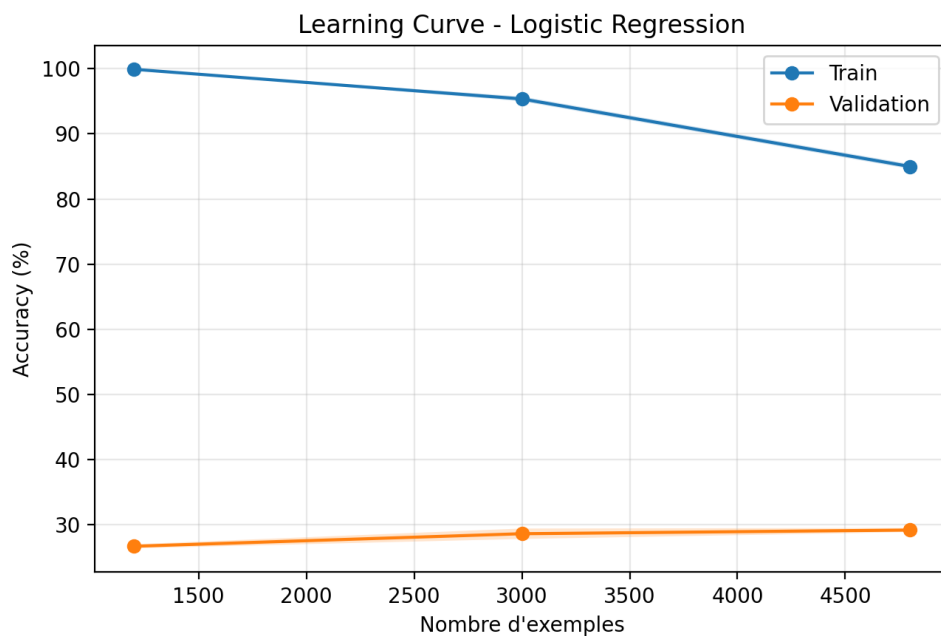


FIGURE IV.2 – Courbe d'apprentissage de la Régression Logistique : accuracy d'entraînement et de validation en fonction du nombre d'exemples.

L'écart important entre les scores d'entraînement (≈ 85 %) et de validation (≈ 27–29 %) traduit un fort **surapprentissage** de la Régression Logistique sur des pixels bruts en haute

dimension.

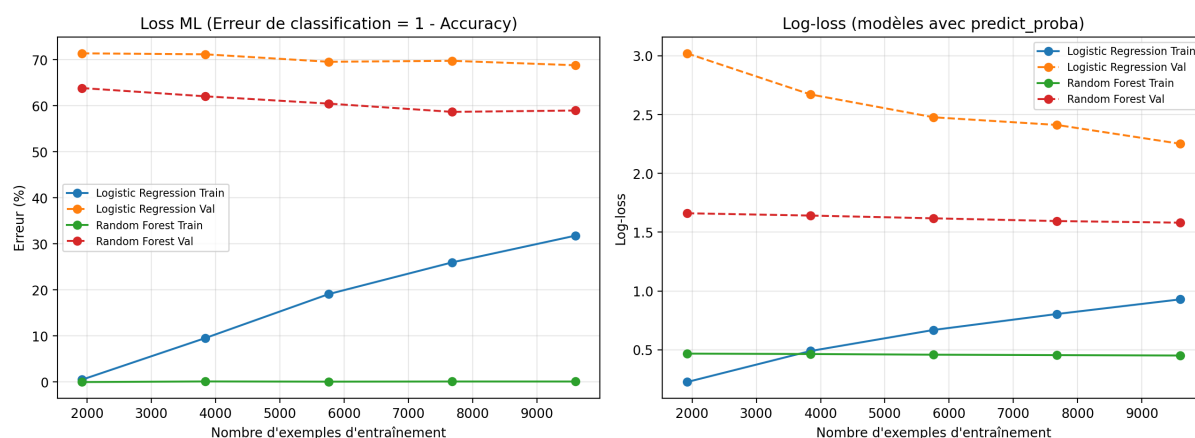


FIGURE IV.3 – Courbes de perte : erreur de classification (1 – accuracy) et log-loss pour LR et RF.

Le Random Forest présente une erreur d'entraînement quasi nulle mais une erreur de validation stable autour de 58–60 %, signe d'un fort surapprentissage ; la Régression Logistique souffre d'un biais élevé avec une erreur de validation ne descendant qu'à $\approx 69\%$.

2.4 Matrices de confusion

Régression Logistique

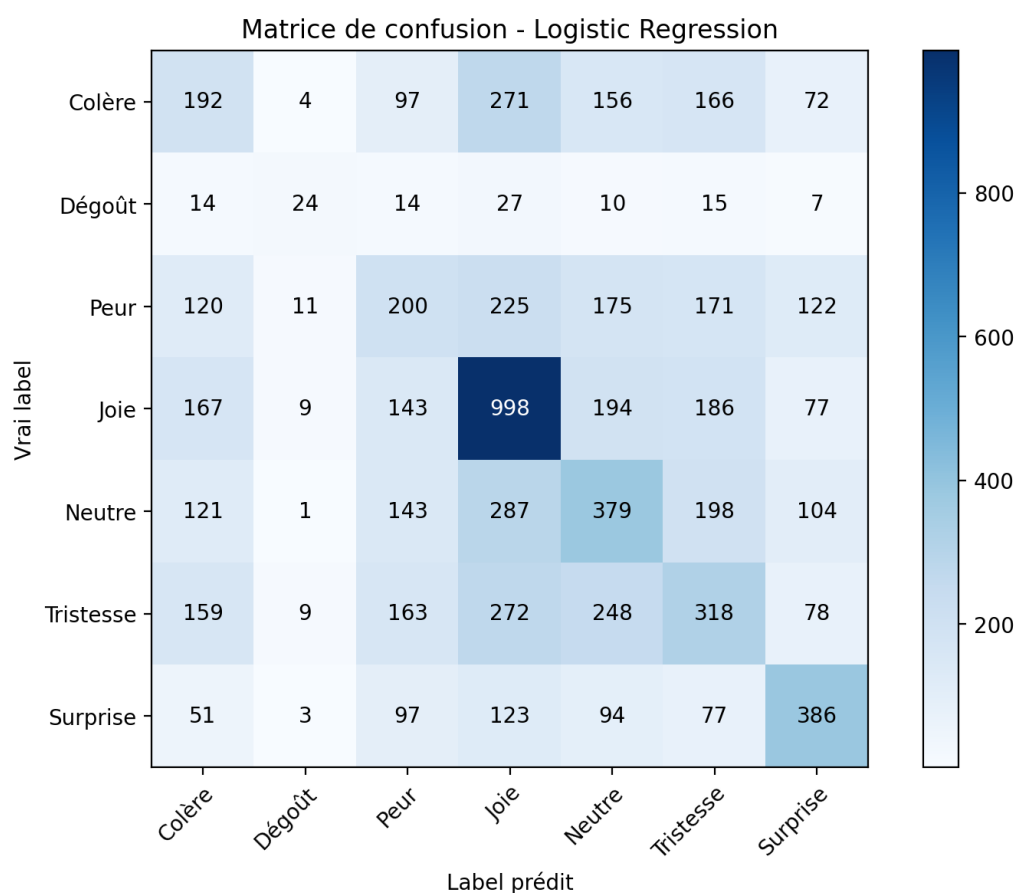


FIGURE IV.4 – Matrice de confusion — Régression Logistique (FER2013, ensemble de test).

La Régression Logistique affiche une accuracy globale d'environ 35 %. La classe *Joie* est la mieux reconnue (998 prédictions correctes), tandis que *Dégoût* est quasi systématiquement mal classifié. Les confusions entre *Peur*, *Colère* et *Tristesse* reflètent l'absence de séparation linéaire dans l'espace pixel.

Random Forest

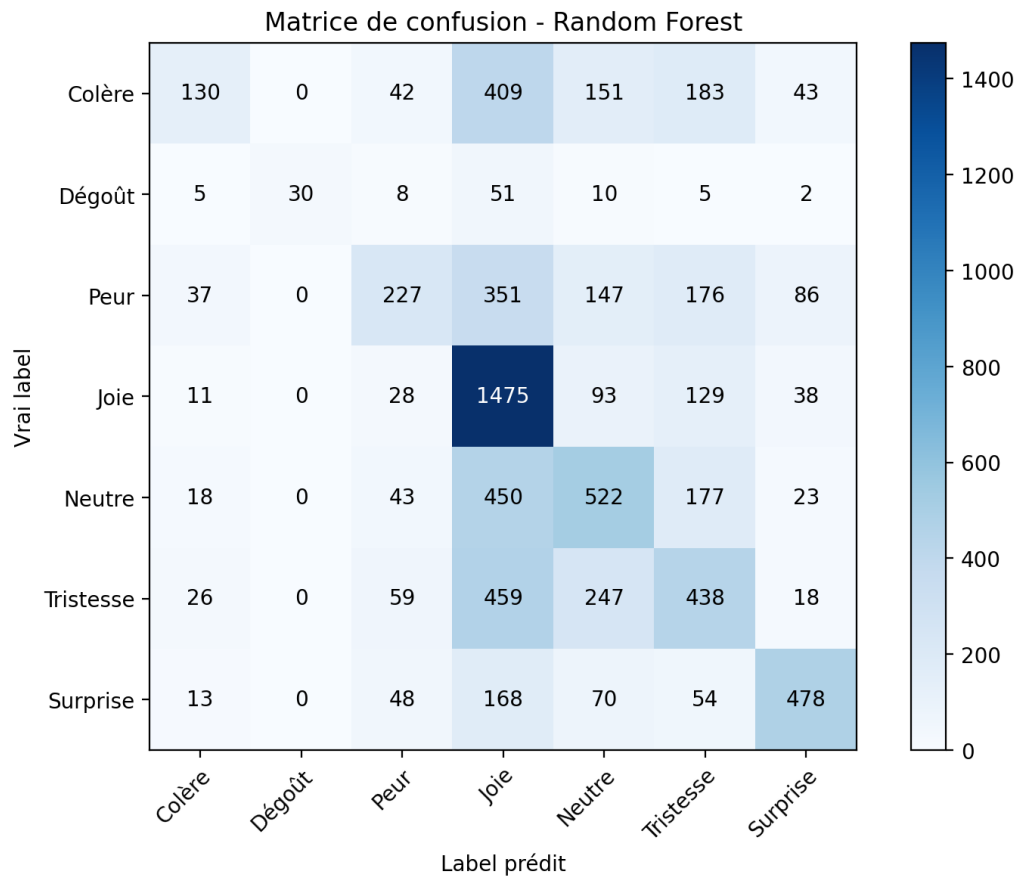


FIGURE IV.5 – Matrice de confusion — Random Forest (FER2013, ensemble de test).

Avec $\approx 46\%$ d'accuracy, le Random Forest présente la diagonale la plus marquée parmi les baselines. Cependant, une large part du gain provient d'une meilleure détection de *Joie* (1 475 correctes), le modèle sur-prédisant massivement cette classe majoritaire.

SVM (noyau RBF)

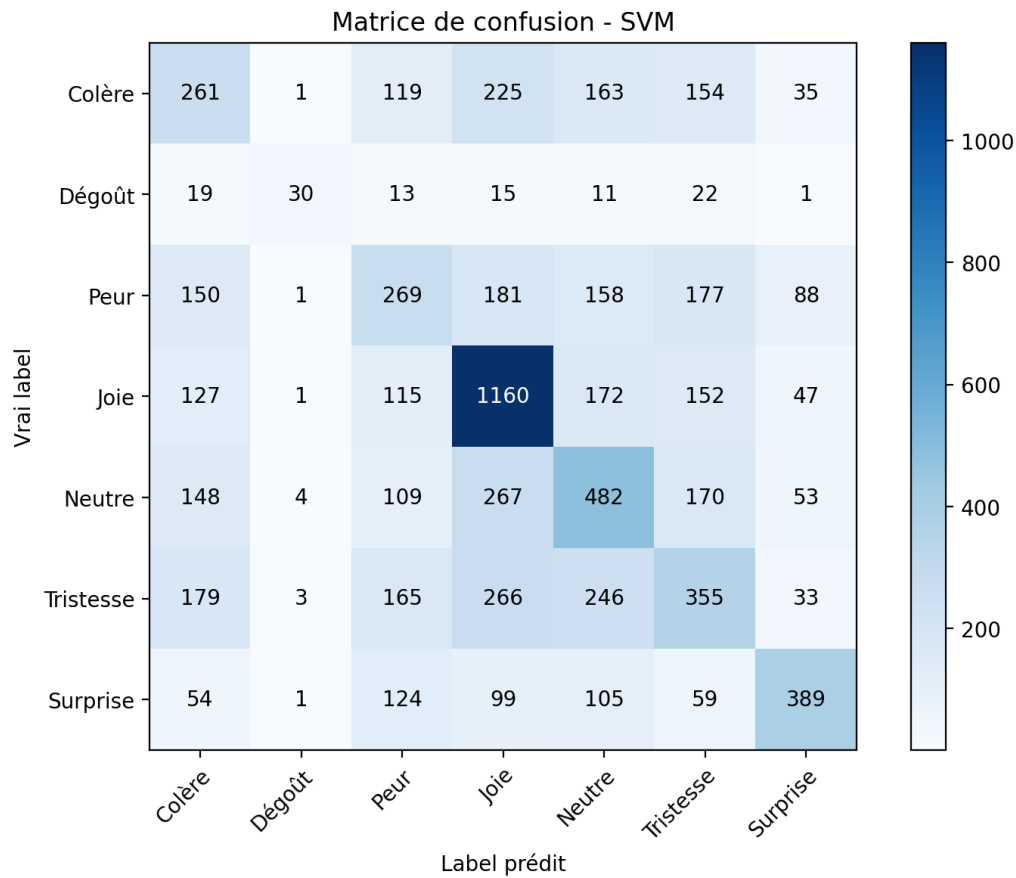


FIGURE IV.6 – Matrice de confusion — SVM noyau RBF (FER2013, ensemble de test).

Le SVM atteint $\approx 41\%$ d’accuracy. Le noyau RBF capture mieux certains patterns non-linéaires — *Surprise* (389 correctes) et *Neutre* (482 correctes) en bénéficient — mais souffre du sous-échantillonnage imposé par la complexité computationnelle $O(n^2)$.

Bilan des baselines ML.. — Quelle que soit l’approche, trois constantes se dégagent : (1) *Joie* est systématiquement la classe la mieux reconnue ; (2) *Dégoût* est quasi impossible à discriminer ; (3) tous les modèles présentent un biais vers *Joie* et *Neutre*, les deux classes majoritaires. Ces résultats confirment que la représentation pixellique est insuffisante pour capturer la structure sémantique des expressions faciales.

3 Custom CNN

3.1 Protocole d’entraînement

Fonction de perte pondérée (Weighted Cross-Entropy)

Pour pallier le déséquilibre des classes, la fonction de perte utilisée est la **Weighted Cross-Entropy Loss** :

$$\mathcal{L}_{\text{WCE}} = -\frac{1}{N} \sum_{i=1}^N w_{y_i} \log \left(\frac{e^{z_{i,y_i}}}{\sum_{k=1}^7 e^{z_{i,k}}} \right)$$

Les poids sont calculés proportionnellement à l'inverse de la fréquence de chaque classe, puis normalisés pour que leur somme soit égale au nombre de classes.

Échantillonnage adaptatif (Weighted Random Sampler)

En complément de la perte pondérée, un **WeightedRandomSampler** ajuste la probabilité de tirage de chaque image lors de la constitution des batches, garantissant que chaque batch présenté au modèle est bien équilibré entre les classes.

Stratégie d'optimisation

L'optimiseur **Adam** est utilisé avec un batch size de 64. Un **ReduceLROnPlateau** réduit le taux d'apprentissage lorsque le F1-macro de validation cesse de progresser. L'entraînement est découpé en **trois sessions reprises** (CustomCNN, CustomCNN_Resume, CustomCNN_Resume-2), permettant d'affiner progressivement le modèle à partir des meilleurs poids sauvegardés. Un mécanisme d'arrêt anticipé (patience = 10) préserve le meilleur checkpoint.

3.2 Analyse des courbes d'apprentissage

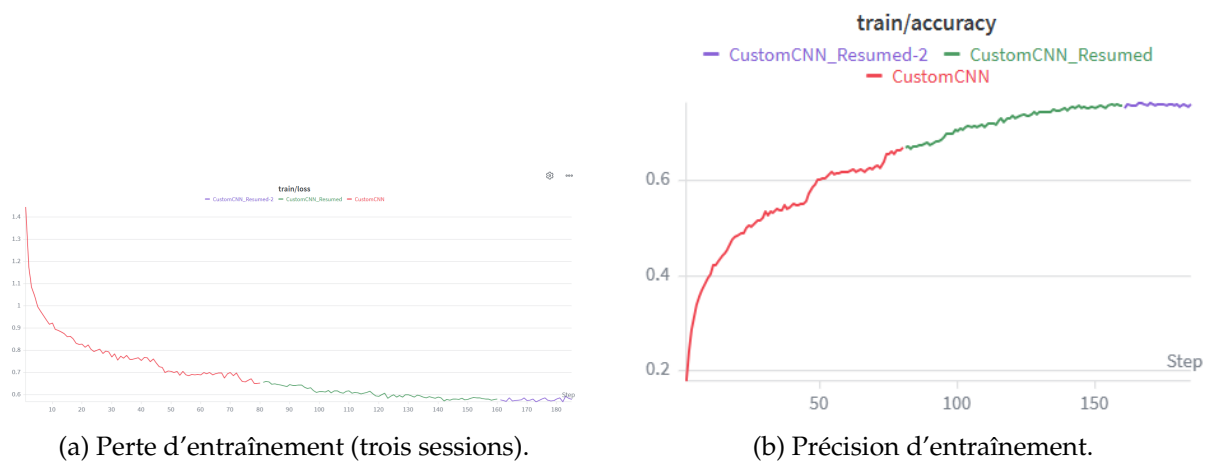


FIGURE IV.7 – Courbes d'entraînement du Custom CNN (trois sessions cumulées).

La perte d'entraînement décroît de façon régulière, passant de ≈ 1.42 à ≈ 0.58 sur l'ensemble des sessions. La précision atteint environ 72–73 % en fin de troisième session, sans instabilité numérique notable.

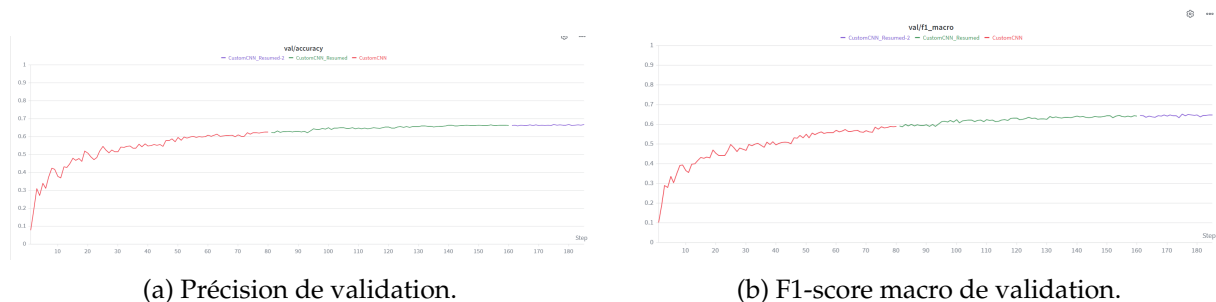


FIGURE IV.8 – Courbes de performance en validation du Custom CNN.

La précision de validation converge vers 66,9 % et le F1-score macro vers 0,651. L'écart modéré avec les métriques d'entraînement (≈ 6 points) traduit un léger surapprentissage, attendu et maîtrisé sur FER2013.

3.3 Métriques finales

best_epoch	175
best_f1	0.65131
best_model_path	best_model.pth
debug/early_stop_counter	10
final_acc	0.66857
final_f1	0.65131

FIGURE IV.9 – Récapitulatif des métriques globales au meilleur epoch (175).

Le meilleur modèle est sauvegardé à l'**époque 175**, avec une précision finale de **66,86 %** et un F1-score macro de **0,6513**.

3.4 Analyse par classe

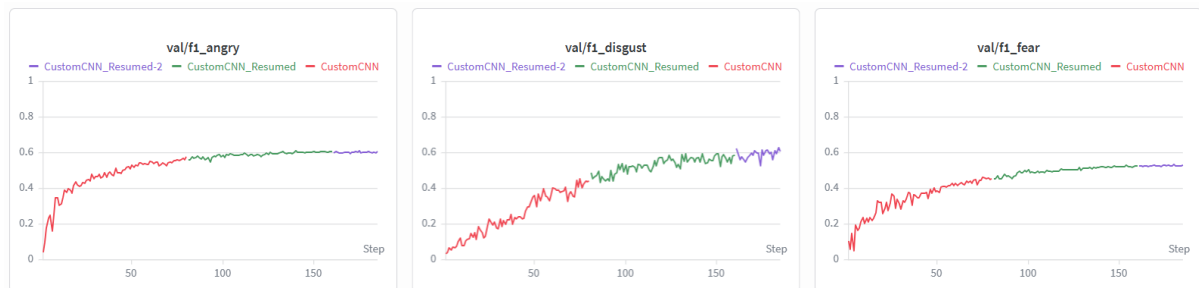


FIGURE IV.10 – F1-score par classe pour *angry*, *disgust* et *fear*.

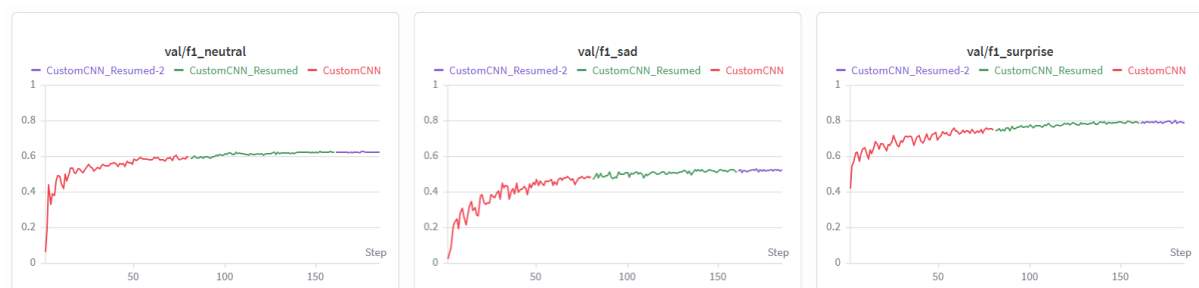
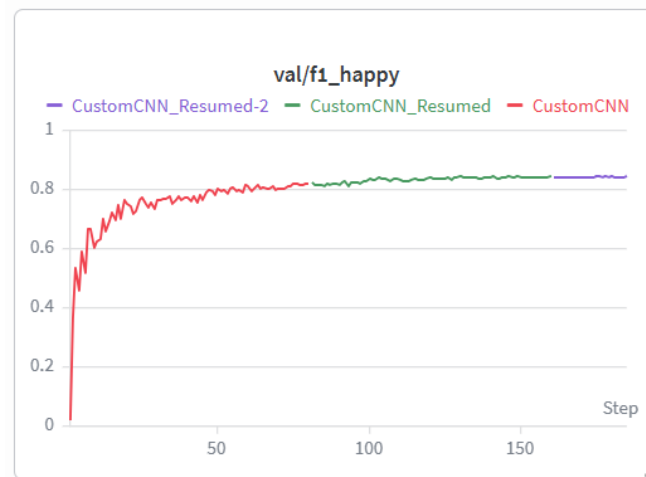


FIGURE IV.11 – F1-score par classe pour *neutral*, *sad* et *surprise*.

FIGURE IV.12 – F1-score pour *happy* — la classe la mieux apprise.

L'observation des courbes par classe révèle une hiérarchie stable : *happy* (≈ 0.85) et *surprise* (≈ 0.80) sont les mieux reconnues ; *fear* et *sad* plafonnent à ≈ 0.50 – 0.53 , confirmant la difficulté intrinsèque à distinguer ces émotions à faible résolution.

3.5 Matrice de confusion

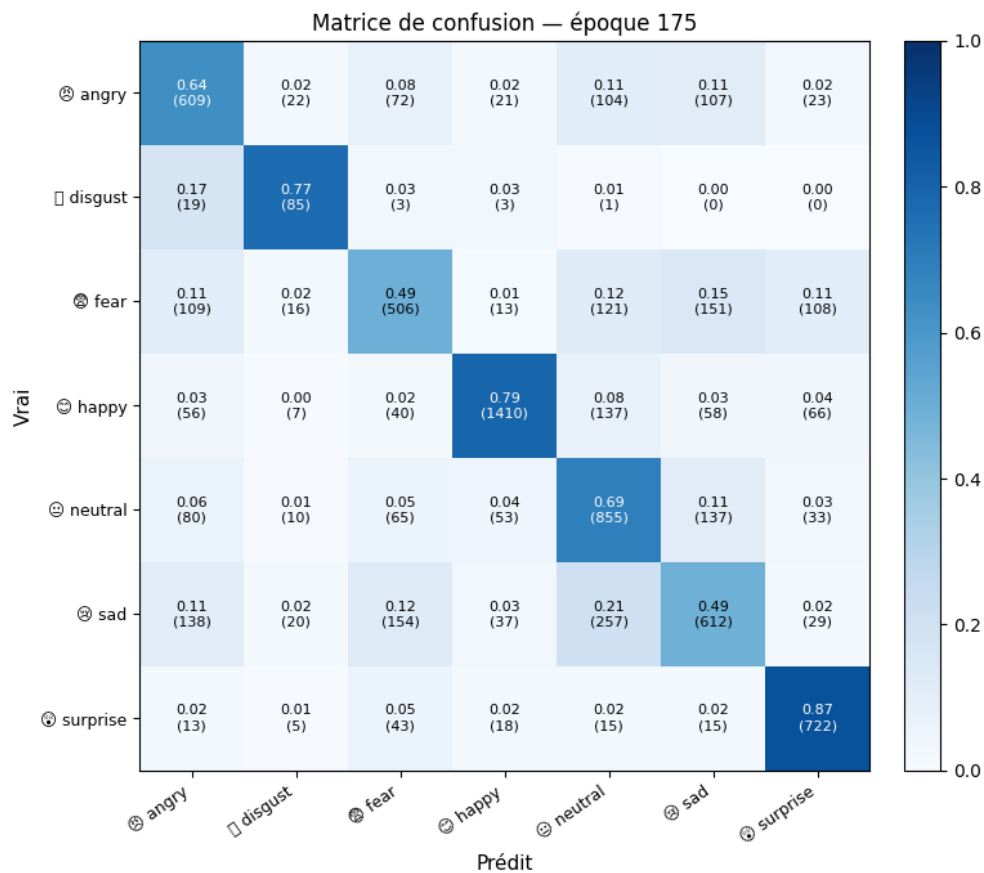


FIGURE IV.13 – Matrice de confusion normalisée du Custom CNN (meilleur checkpoint, époque 175).

Par rapport aux baselines ML, la diagonale est nettement plus marquée : le rappel moyen sur les 7 classes passe de $\approx 32\text{--}40\%$ à $\approx 66\%$. Le biais vers *happy* est fortement réduit grâce à la pondération et à l’échantillonnage adaptatif, et le rappel de *disgust* passe de quasi nul à 0,77 — le gain le plus spectaculaire entre les deux approches.

4 Transfer Learning — ResNet50

4.1 Présentation et configuration

ResNet50 est un réseau résiduel de 50 couches comptant $\approx 24,6\text{M}$ paramètres [4]. Ses *skip connections* permettent l’entraînement de réseaux profonds sans dégradation des performances. Comparé à VGG16 ($\approx 138\text{M}$ paramètres), il offre un meilleur compromis capacité/coût, le rendant plus adapté à un dataset de taille modérée comme FER2013.

TABLE IV.3 – Hyperparamètres de l’entraînement ResNet50

Paramètre	Valeur
Poids initiaux	ImageNet (préentraîné)
Optimiseur	Adam
Batch size	96
Dropout	0,5
Scheduler	Cosine annealing
Nombre total d’époques	46 (3 phases)

4.2 Stratégie d’entraînement par phases

Phase 1 — Échauffement (époques 1–15).. — Seule la tête de classification est entraînable. La loss descend de $\approx 1,80$ à $\approx 1,62$. Meilleur val F1 : **0,3505**.

Phase 2 — Fine-tuning de layer4 (époques 16–25).. — `layer4` est dégelée. Amélioration immédiate : val accuracy autour de 0,41–0,42. Meilleur val F1 : **0,3700** (+0,020 vs Phase 1).

Phase 3 — Fine-tuning de layer3 (époques 26–46).. — `layer3` est dégelée à son tour. Phase la plus productive : val accuracy pic à **0,4449** à l’époque 36. Meilleur val F1 : **0,3985**.

4.3 Courbes d'apprentissage et résultats

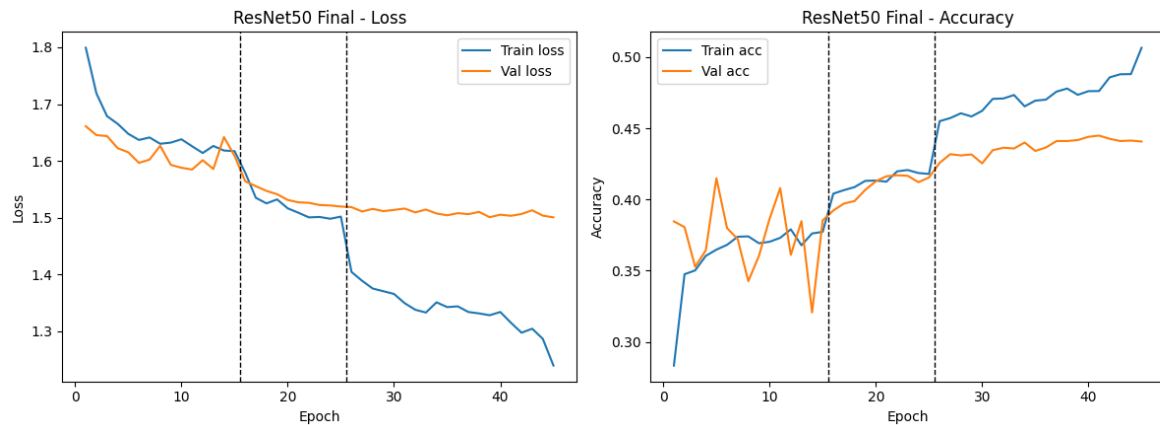


FIGURE IV.14 – Courbes de loss et d'accuracy de ResNet50 sur FER2013. Les lignes pointillées délimitent les trois phases.

TABLE IV.4 – Métriques globales de ResNet50 sur le jeu de test

Métrique	Valeur
Test Accuracy	44,36 %
Test Macro F1	0,3986
Val Macro F1 (best)	0,3985

La quasi-parfaite cohérence entre val F1 (0,3985) et test F1 (0,3986) confirme l'absence de surapprentissage sur le jeu de validation.

4.4 Analyse par classe et matrice de confusion

```

Test accuracy: 0.4436
Test macro F1: 0.3986

Classification report:

```

	precision	recall	f1-score	support
angry	0.3796	0.2797	0.3221	958
disgust	0.1140	0.4414	0.1811	111
fear	0.2746	0.2344	0.2529	1024
happy	0.7147	0.5620	0.6292	1774
neutral	0.3747	0.5896	0.4582	1233
sad	0.4195	0.3007	0.3503	1247
surprise	0.5623	0.6354	0.5966	831
accuracy			0.4436	7178
macro avg	0.4056	0.4348	0.3986	7178
weighted avg	0.4706	0.4436	0.4460	7178

FIGURE IV.15 – Rapport de classification complet de ResNet50 (accuracy = 44,36 %, macro F1 = 0,3986).

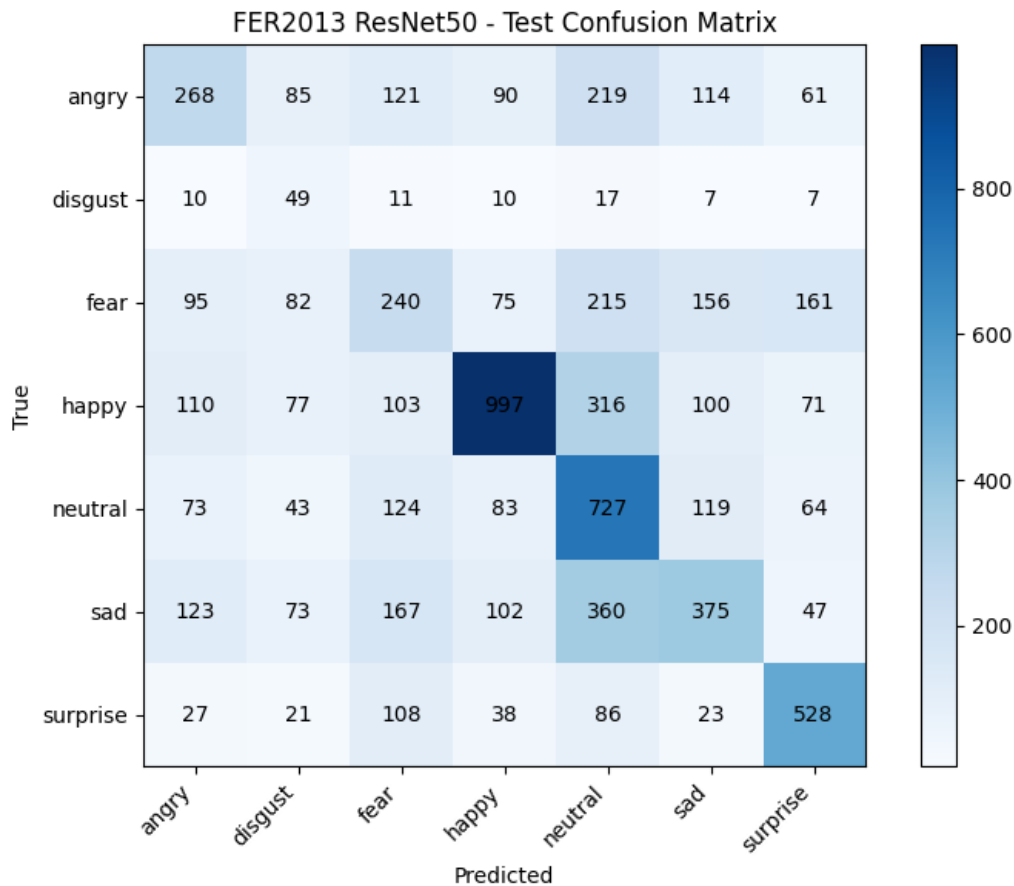


FIGURE IV.16 – Matrice de confusion de ResNet50 sur le jeu de test FER2013.

Happy ($F1 = 0,629$) et *Surprise* ($F1 = 0,597$) sont les mieux reconnues. *Neutral* ($F1 = 0,458$) joue un rôle de classe refuge — sa précision faible ($0,375$) révèle que le modèle y redirige les exemples ambigus. *Disgust* ($F1 = 0,181$) souffre du déséquilibre extrême de sa classe (111 exemples de test), malgré la pondération.

Bien que ResNet50 soit nettement plus profond (50 couches, 24,6M params), ses performances restent inférieures au CustomCNN (44,4 % vs 66,9 %). L'écart s'explique principalement par l'inadéquation du domaine de préentraînement (images RGB 224×224) avec les contraintes de FER2013 (niveaux de gris, 48×48).

5 Transfer Learning — VGG16

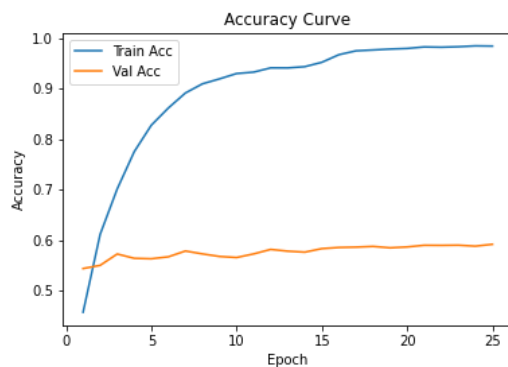
5.1 Présentation et configuration

VGG16 est un réseau de 16 couches développé par l'Université d'Oxford [5], avec environ 138 millions de paramètres. Son architecture homogène (filtres 3×3 , max-pooling 2×2) en fait un modèle de référence pour le transfer learning.

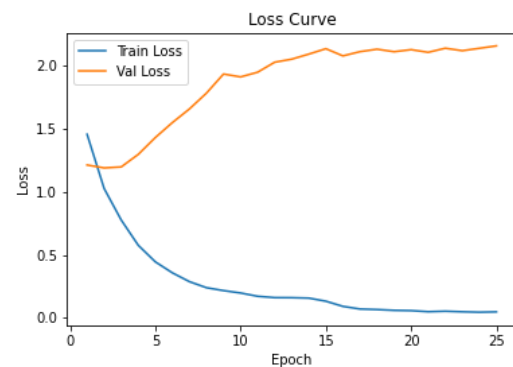
TABLE IV.5 – Hyperparamètres de l'entraînement VGG16

Paramètre	Valeur
Poids initiaux	ImageNet (préentraîné)
Optimiseur	Adam
Taux d'apprentissage	10^{-4}
Batch size	32
Nombre d'époques	30
Régularisation	Dropout (couches FC)

5.2 Courbes d'apprentissage et analyse



(a) Courbe d'accuracy (Train vs Validation)



(b) Courbe de perte (Train vs Validation)

FIGURE IV.17 – Courbes d'entraînement de VGG16 sur FER2013 — surapprentissage marqué.

Les courbes révèlent un surapprentissage très prononcé : la précision d'entraînement monte à quasi 99 % tandis que la validation stagne autour de 58–59 %. La validation loss amorce une remontée après quelques époques, signalant que les capacités du modèle sont disproportionnées par rapport à la taille de FER2013. Une stratégie d'early stopping aurait permis de conserver un meilleur checkpoint.

5.3 Matrice de confusion

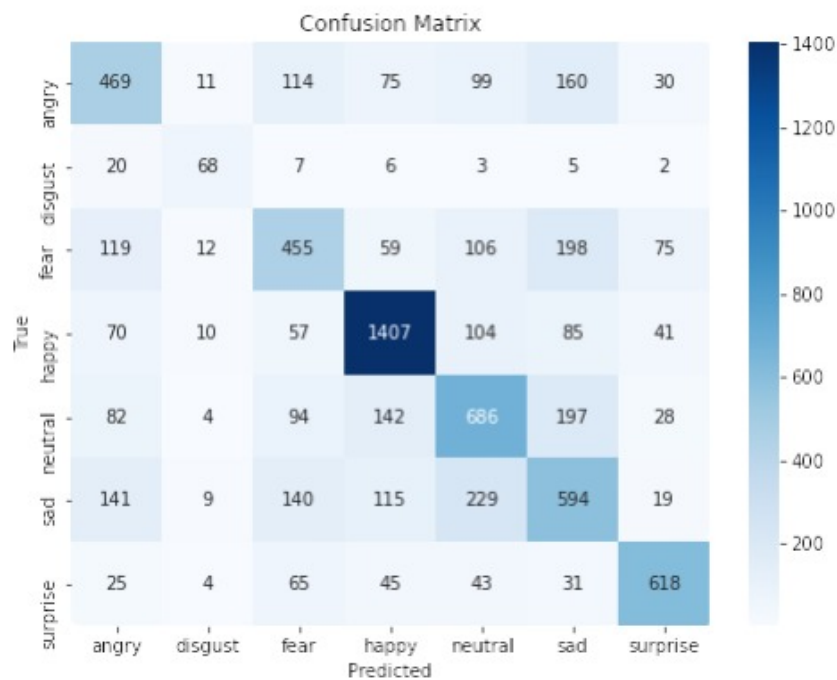


FIGURE IV.18 – Matrice de confusion de VGG16 sur le jeu de test FER2013.

VGG16 présente des valeurs diagonales plus élevées que ResNet50 en valeur absolue (*happy* : 1 407 ; *surprise* : 618 ; *angry* : 469), cohérent avec son accuracy de validation supérieure. Toutefois, le fort surapprentissage documenté relativise ces résultats, et le CustomCNN reste largement en tête malgré ses 24 fois moins de paramètres.

5.4 Limites et recommandations

Les principales limites identifiées pour VGG16 dans ce contexte sont la capacité excessive du modèle (138M paramètres pour $\approx 28\,000$ images d'entraînement), le gel insuffisant des couches basses aggravant le surapprentissage, et la dégradation introduite par l'interpolation de 48×48 vers 224×224 .

Pour améliorer les performances, il serait pertinent d'adopter une architecture plus légère (MobileNetV2, EfficientNet-B0), d'implémenter un early stopping basé sur la validation loss, et de geler les blocs convolutifs inférieurs en n'affinant que les dernières couches.

CHAPITRE V

Interface de Démonstration Interactive

1 Motivation et Objectifs

L'entraînement et l'évaluation des modèles constituent le cœur scientifique du projet, mais la valeur pratique d'un système de reconnaissance d'émotions ne se manifeste pleinement que lorsqu'un utilisateur peut l'interroger directement sur des images de son choix. C'est dans cette perspective qu'une interface interactive a été développée à l'aide du framework **Streamlit**, permettant de tester l'ensemble des modèles — classiques et profonds — sans qu'aucune ligne de code ne soit nécessaire côté utilisateur.

Les objectifs de l'interface sont les suivants :

- Permettre la comparaison directe entre les approches ML classiques (`.pkl`) et les modèles de Deep Learning (`.pth`) sur une même image.
- Exposer les probabilités brutes par classe pour analyser la distribution de confiance du modèle.
- Offrir un environnement de test reproductible pour valider les résultats présentés dans ce rapport.
- Documenter le pipeline d'inférence complet, de l'image brute au vecteur de probabilités.

2 Architecture de l'Application

L'application est structurée en deux zones fonctionnelles distinctes, illustrées dans la figure V.1.

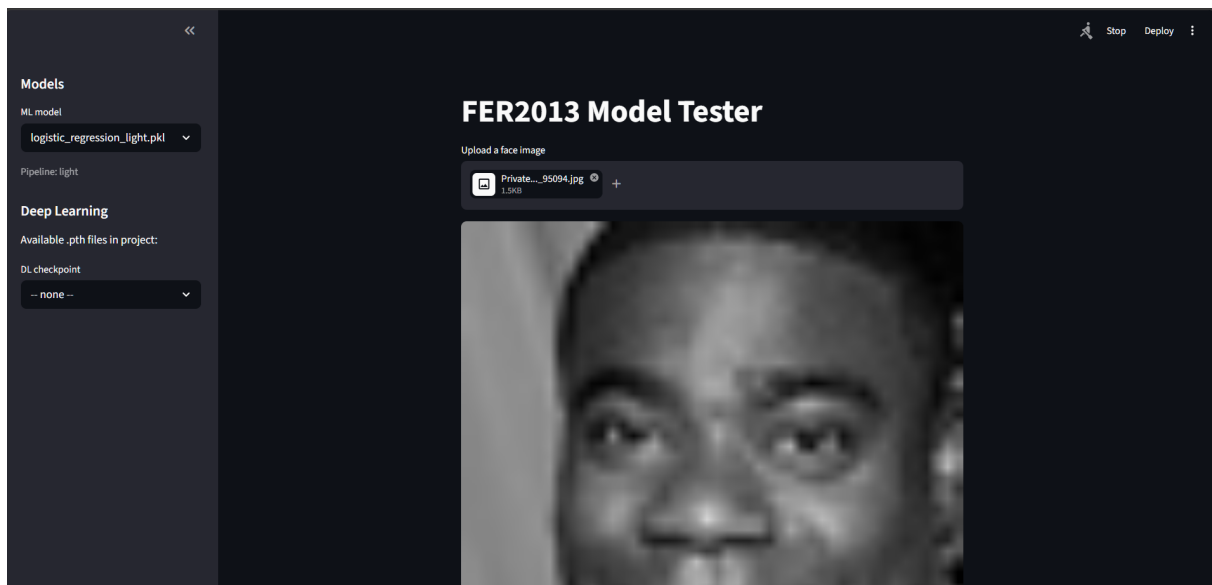


FIGURE V.1 – Vue d’ensemble de l’interface Streamlit. À gauche : panneau de sélection des modèles. À droite : zone principale d’inférence.

2.1 Panneau latéral — Sélection des modèles

La barre latérale expose deux groupes de contrôles indépendants :

Modèles ML classiques.. — Un menu déroulant liste automatiquement tous les fichiers `.pkl` présents dans le dossier `saved_models/`. Le nom de fichier encode à la fois le type de modèle et le pipeline de prétraitement utilisé à l’entraînement (ex. `random_forest_light.pkl`); l’application extrait ces métadonnées par analyse du nom pour appliquer automatiquement le pipeline correspondant lors de l’inférence.

Modèles Deep Learning.. — Un second menu liste les fichiers `.pth` et `.pt` présents à la racine du projet. L’architecture correspondante (CustomCNN, VGG16, ResNet50) est détectée automatiquement à partir des noms de paramètres du *checkpoint*, sans intervention de l’utilisateur.

2.2 Zone principale — Inférence et résultats

La zone centrale reçoit l’image uploadée et présente les résultats en trois blocs successifs :

1. **Prévisualisation** de l’image originale.
2. **Prédiction** : étiquette de classe prédite et nom du modèle.
3. **Distribution de probabilités** : vecteur de confiance affiché au format JSON interactif, permettant d’observer la distribution complète sur les 7 classes.

3 Pipeline d’Inférence

L’inférence emprunte deux chemins distincts selon la nature du modèle sélectionné, décrits ci-après et schématisés en figure V.2.

3.1 Chemin ML classique

1. Chargement du modèle via `joblib.load()`.
2. Conversion de l’image en niveaux de gris et redimensionnement à 48×48 .

3. Application du pipeline de prétraitement associé au modèle (`minimal`, `light` ou `aggressive`).
4. Normalisation Z-score ($\mu = 0,5071$, $\sigma = 0,2520$) et aplatissement en vecteur de 2 304 features.
5. Appel à `model.predict()` et, si disponible, à `model.predict_proba()` pour les probabilités.

3.2 Chemin Deep Learning

1. Chargement du *checkpoint* via `torch.load(map_location="cpu")`.
2. **Extraction automatique du *state dict*** : le chargeur teste successivement les clés `model_state_dict`, `state_dict`, `model_state`, puis vérifie si le dictionnaire racine contient directement des tenseurs.
3. **Détection automatique de l'architecture** par analyse des préfixes des clés du *state dict* :
 - `features.*` $\Rightarrow$ VGG16
 - `layer1.*` $\Rightarrow$ ResNet50
 - `block1.*` $\Rightarrow$ CustomCNN
4. Reconstruction du modèle avec l'architecture exacte utilisée à l'entraînement et chargement des poids.
5. Prétraitement adapté au modèle détecté :
 - **CustomCNN** : `grayscale` $\rightarrow 48 \times 48 \rightarrow$ pipeline `light` $\rightarrow$ tenseur $(1, 1, 48, 48)$.
 - **VGG16 / ResNet50** : `grayscale` $\rightarrow 48 \times 48 \rightarrow 224 \times 224 \rightarrow 3$ canaux $\rightarrow$ normalisation ImageNet $\rightarrow$ tenseur $(1, 3, 224, 224)$.
6. Passage en mode évaluation (`model.eval()`), inférence sous `torch.no_grad()`, et application de `softmax` pour obtenir les probabilités.

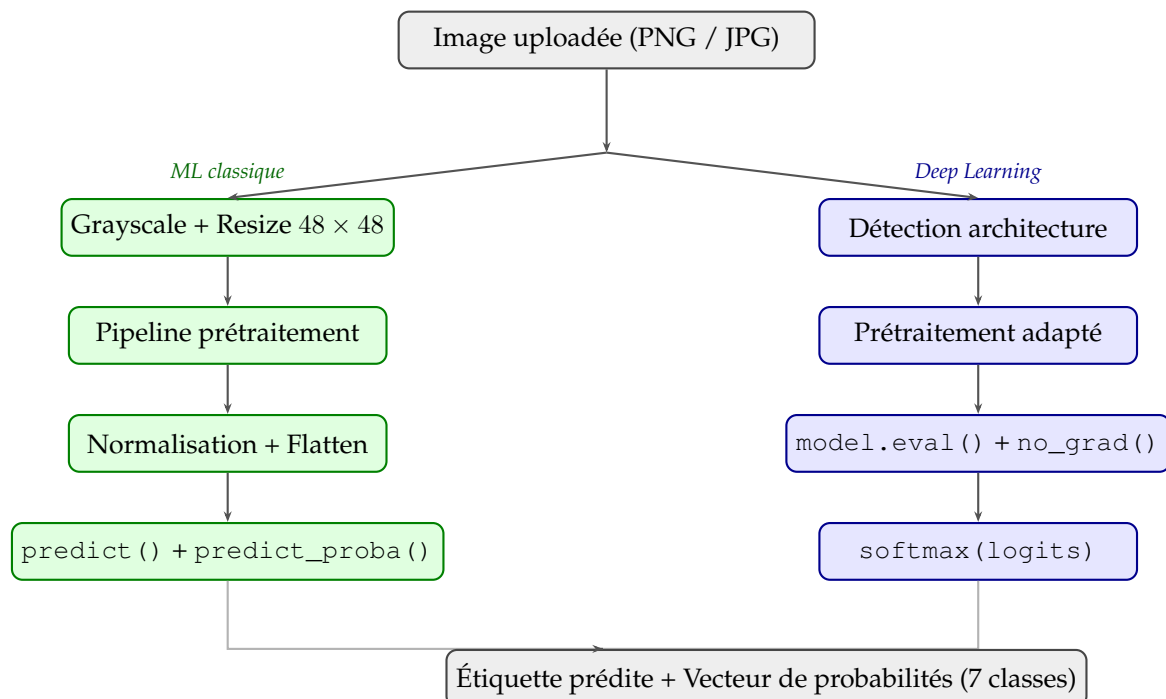


FIGURE V.2 – Schéma du pipeline d'inférence de l'interface Streamlit pour les deux familles de modèles.

4 Chargement Robuste des Checkpoints

Un soin particulier a été apporté au chargement des fichiers `.pth`, dont le format varie selon la façon dont le checkpoint a été sauvegardé lors de l'entraînement. L'application gère de manière transparente les trois formats courants :

Listing V.1 – Extraction robuste du state dict depuis différents formats de checkpoint.

```

1 def _extract_state_dict(obj):
2     if not isinstance(obj, dict):
3         return obj # state dict sauvegarde directement
4     for key in ("model_state_dict", "state_dict", "model_state", "model"):
5         if key in obj and isinstance(obj[key], dict):
6             return obj[key]
7     # Le dict racine contient directement les tenseurs
8     if any(isinstance(v, torch.Tensor) for v in obj.values()):
9         return obj
10    return None

```

La détection de l'architecture repose ensuite sur les préfixes des clés du *state dict*, une approche plus fiable que l'inspection du fichier brut, car elle est indépendante du format d'encapsulation :

Listing V.2 – Détection automatique de l'architecture à partir des clés du state dict.

```

1 keys      = list(state.keys())
2 is_vgg    = any(k.startswith("features.") for k in keys)
3 is_resnet = any(k.startswith("layer1.")  for k in keys)
4 is_cnn    = any(k.startswith("block1.")  for k in keys)

```

En cas d'échec du chargement strict (`strict=True`), l'application tente un chargement non-strict (`strict=False`) et affiche un avertissement explicite. Un panneau de débogage rétractable expose les noms et dimensions des paramètres clés du checkpoint, facilitant le diagnostic en cas d'inadéquation architecturale.

5 Exemples d'Inférence

Les figures suivantes illustrent le comportement de l'interface sur des cas représentatifs.

5.1 Cas favorable — Émotion nettement exprimée

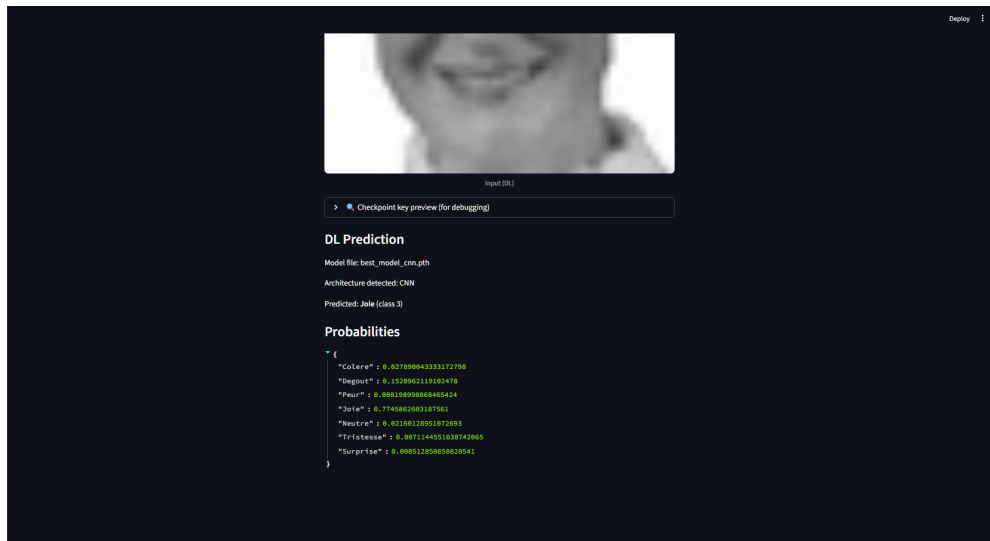


FIGURE V.3 – Inférence sur une image de *Joie*. Le CustomCNN attribue une probabilité de $\approx 0,77$ à la classe correcte, reflétant les résultats de la matrice de confusion ($F1 \approx 0,85$).

5.2 Cas ambigu — Peur

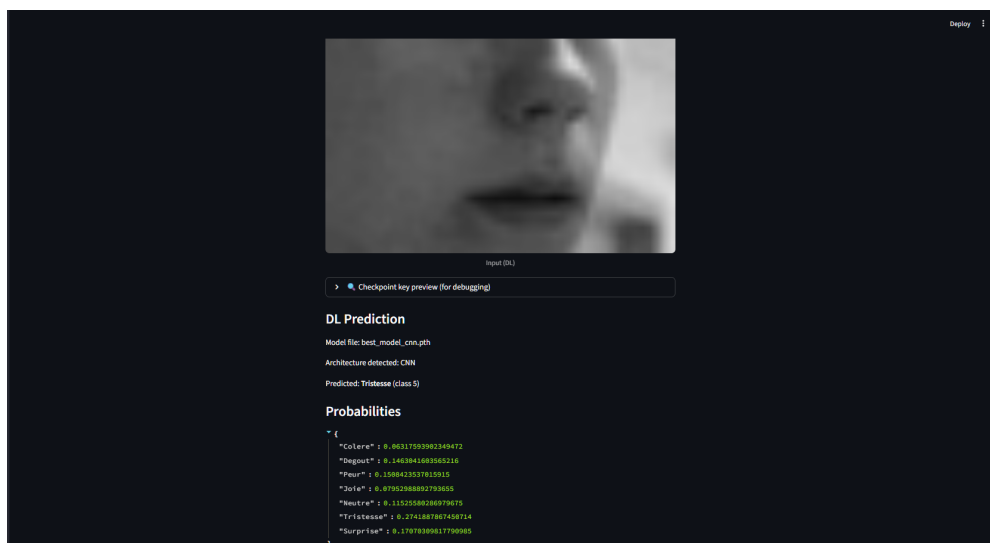


FIGURE V.4 – Inférence sur une image de *Peur*. La probabilité est distribuée entre *Peur*, *Tristesse*, *Dégout* et *Surprise* ($\approx 0,35$), illustrant la confusion inter-classes documentée au chapitre ??.

5.3 Comparaison ML vs Deep Learning

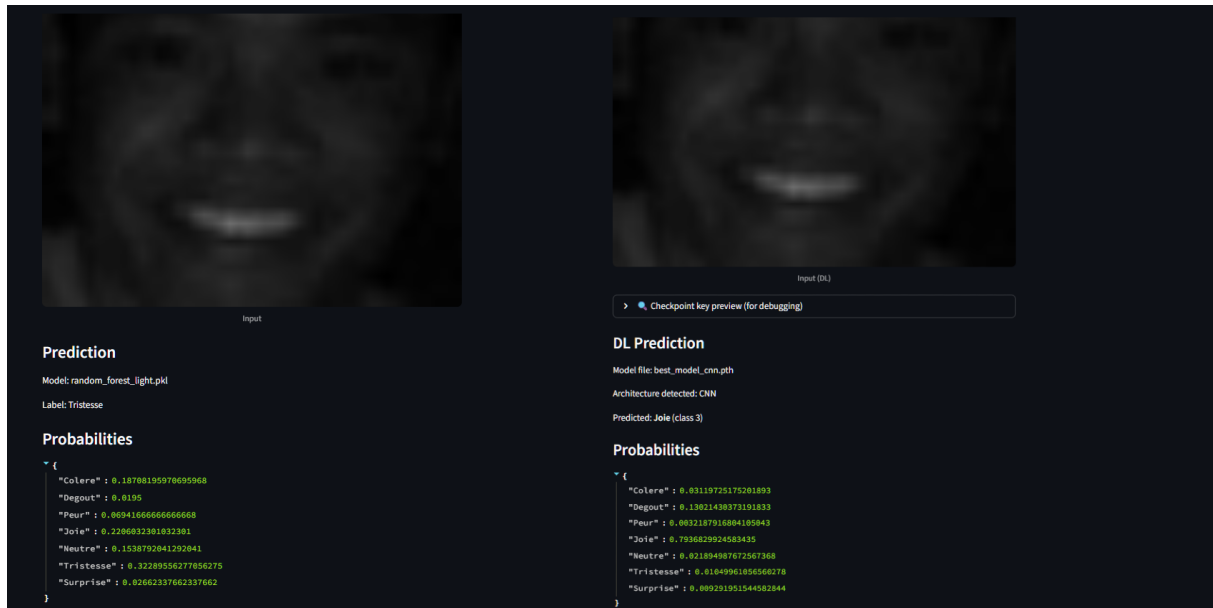


FIGURE V.5 – Même image soumise simultanément à un modèle ML (Random Forest à gauche) et au CustomCNN (à droite). Le modèle ML prédit incorrectement *Tristesse* avec $\approx 0,32$ de confiance, tandis que le CNN identifie correctement l'émotion cible (*Joie*).

6 Bilan Technique

Fonctionnalité	Description	Statut
Inférence ML classique	Chargement .pkl + prétraitement adaptatif	Opérationnel
Inférence Deep Learning	Chargement .pth multi-format	Opérationnel
Détection d'architecture	Automatique par analyse des clés	Opérationnel
Probabilités par classe	Vecteur complet sur 7 émotions	Opérationnel
Pipeline adaptatif	minimal / light / aggressive	Opérationnel
Débogage checkpoint	Panneau rétractable avec dimensions	Opérationnel
Flux vidéo temps réel	Webcam + détection faciale en direct	Non implémenté
Visualisation Grad-CAM	Carte d'activation pour interprétabilité	Non implémenté

Les deux fonctionnalités manquantes — flux webcam et Grad-CAM — constituent des objectifs avancés identifiés dès le cadrage (chapitre I, §2.3). Leur intégration représente la principale perspective d'extension de l'interface.

CHAPITRE VI

Bilan Comparatif et Conclusion

1 Synthèse comparative des performances

TABLE VI.1 – Synthèse comparative des performances sur FER2013

Famille	Modèle	Accuracy (%)	F1-macro
ML classique	Logistic Regression	≈ 35	$\approx 0,33$
ML classique	SVM (RBF)	≈ 41	$\approx 0,41$
ML classique	Random Forest	≈ 46	$\approx 0,49$
Deep Learning	ResNet50 (TL)	44,36	0,399
Deep Learning	VGG16 (TL)	$\approx 58-59$	—
Deep Learning	Custom CNN	66,86	0,651

2 Analyse critique par famille d’approches

Modèles ML classiques

Les baselines confirment que la représentation pixellique est insuffisante pour capturer la structure sémantique des expressions faciales. Random Forest offre la meilleure accuracy parmi eux ($\approx 46\%$) avec un temps d’entraînement très court, mais son surapprentissage marqué limite la généralisation. La Régression Logistique souffre d’un biais structurel face à un espace non linéairement séparable, et le SVM est pénalisé par le coût computationnel prohibitif imposant un sous-échantillonnage.

Apprentissage profond

Le **Custom CNN** (5,87M paramètres) s’impose comme le meilleur modèle avec 66,86 % d’accuracy et un F1-macro de 0,651. L’architecture pyramidale combinée à la Batch Normalization, au Dropout, à la pondération des classes et à une stratégie d’entraînement incrémental sur trois sessions démontre qu’un réseau léger entraîné *from scratch* peut surpasser des architectures de transfer learning bien plus lourdes lorsqu’il est correctement régularisé et adapté à la tâche.

ResNet50 (24,6M paramètres) atteint 44,36 % avec un F1-macro de 0,399. Le dégel progressif en trois phases a apporté des gains cumulés de +0,048 en F1, mais le domaine de préentraînement (images RGB haute résolution) reste partiellement inadapté aux visages en niveaux de gris 48×48 .

VGG16 (138M paramètres) plafonne à $\approx 58\text{--}59\%$ en validation malgré une précision d’entraînement proche de 99% , illustrant le risque d’un transfer learning mal calibré — la taille du modèle doit être proportionnée aux données disponibles.

3 Enseignements transversaux

Plusieurs leçons généralisables émergent de cette étude :

1. **La qualité de la représentation est déterminante.** Le passage des pixels bruts aux features convolutives constitue le gain le plus significatif observé, surpassant toute optimisation d’hyperparamètres.
2. **La taille du modèle doit être proportionnée aux données disponibles.** VGG16, conçu pour ImageNet (1,2M images), souffre d’un espace d’hypothèses disproportionné face à FER2013.
3. **Le déséquilibre de classes reste un défi structurel.** *Fear* et *sad* constituent les points faibles communs à tous les modèles, en raison de leur proximité visuelle et du bruit de labellisation inhérent à FER2013 ($\approx 15\%$).

4 Perspectives

Pour dépasser le plafond actuel, plusieurs pistes prometteuses ont été identifiées : utilisation de backbones modernes mieux adaptés aux petites images (EfficientNet-B4, Vision Transformer), augmentation ciblée sur les classes difficiles, label smoothing pour atténuer le bruit d’annotation, et méthodes d’ensemble combinant les forces de chaque architecture. L’intégration de descripteurs complémentaires (points de repère faciaux, Action Units FACS) constitue également une voie d’amélioration significative au-delà des seules approches bout-en-bout.

Ce projet aura permis de maîtriser l’ensemble de la chaîne d’un système de reconnaissance d’émotions : de la constitution et l’analyse du dataset jusqu’au déploiement et à l’évaluation comparative des modèles, en passant par le prétraitement et la conception architecturale. Les résultats obtenus constituent une base solide pour des travaux futurs dans ce domaine.

Bibliographie

- [1] I.J. Goodfellow et al., "Challenges in Representation Learning : A Report on Three Machine Learning Contests," ICML 2013.
- [2] A. Mollahosseini et al., "AffectNet : A Database for Facial Expression Recognition in the Wild," IEEE Transactions on Affective Computing, 2019.
- [3] P. Ekman & W.V. Friesen, "Constants Across Cultures in the Face and Emotion," Journal of Personality and Social Psychology, 1971.
- [4] K. He et al., "Deep Residual Learning for Image Recognition," CVPR 2016.
- [5] K. Simonyan & A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," ICLR 2015.
- [6] A. Howard et al., "MobileNets : Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017.
- [7] Kaggle — FER2013 Dataset : <https://www.kaggle.com/datasets/msmbare/fer2013>.